CSA06 — Design and Analysis of Algorithms

Analytical Study and Implementation of Brute Force, Divide-and-Conquer, and Hybrid Algorithms for the Closest Pair and Convex Hull Problems

Name	Jeeva
Register No.	192421334
Course Outcome (CO)	CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull
Bloom's Taxonomy	L4 - Analyse, L5 - Evaluate, L6 - Create
SDG Mapping	SDG 4 - Quality Education; SDG 9 - Industry, Innovation and Infrastructure

This report is presented in two parts. **Part A** solves the assignment's worked example (the given 10-point dataset) by hand/computation using Brute Force, as required for the core problem statement. **Part B** extends the study to the assignment's full constraints: a larger real-world-style coordinate dataset (10^3-10^6 points), all three algorithm families (Brute Force, Divide-and-Conquer, and Hybrid) implemented and benchmarked in the same environment, with execution time, complexity analysis, graphs, assumptions, limitations and a comparative discussion.

Contents

Part A — Worked Example on the Given 10-Point Dataset

Point set P = {A(2,3), B(5,8), C(9,4), D(12,10), E(7,2), F(3,11), G(15,6), H(10,14), I(6,12), J(1,7)}.

A.1 Problem analysis

Two classic computational-geometry problems are solved with Brute Force: (i) **Closest Pair** - find the two points of minimum Euclidean distance by examining every possible pair; (ii) **Convex Hull** - find the smallest convex polygon enclosing all points, by testing every candidate edge against every other point's orientation.

A.2 Brute Force Closest Pair — algorithm

```
CLOSEST-PAIR-BF(P[1..n]):
  minDist ← ∞
  for i ← 1 to n-1:
    for j ← i+1 to n:
      d ← sqrt((P[i].x-P[j].x)^2 + (P[i].y-P[j].y)^2)
      if d < minDist: minDist ← d ; closestPair ← (P[i],P[j])
  return closestPair, minDist
```

A.3 Step-by-step calculation (representative distances)

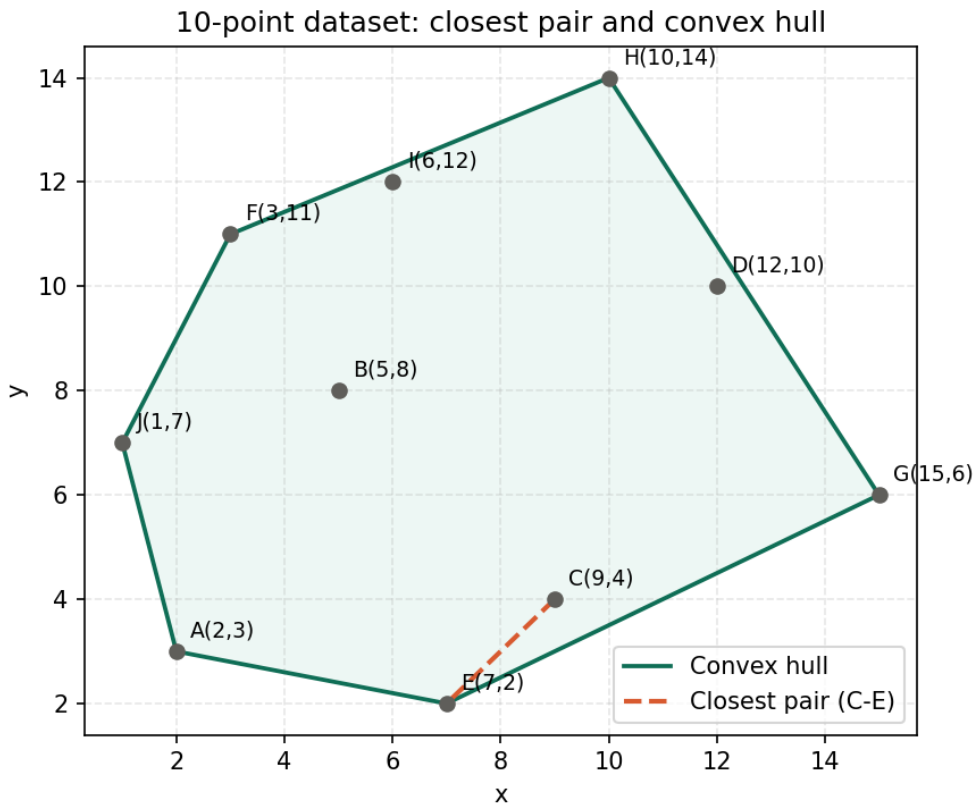
Pair	Δx	Δy	Distance
C(9,4) - E(7,2)	2	2	√8 = 2.828 (minimum)
I(6,12) - F(3,11)	3	1	√10 = 3.162
B(5,8) - F(3,11)	2	3	√13 = 3.606
A(2,3) - J(1,7)	1	4	√17 = 4.123
D(12,10) - H(10,14)	2	4	√20 = 4.472

All C(10,2) = 45 pairwise distances were computed and compared (verified computationally); the table above lists only the smallest for illustration. **Result: closest pair = C(9,4) and E(7,2), minimum distance = 2√2 ≈ 2.83 units.**

A.4 Brute Force Convex Hull — algorithm

```
CONVEX-HULL-BF(P[1..n]):
  for i ← 1 to n:
    for j ← 1 to n, j ≠ i:
      pos ← 0 ; neg ← 0
      for k ← 1 to n, k≠i, k≠j:
        cross ← (Pj-Pi) × (Pk-Pi)
        if cross>0: pos++ else if cross<0: neg++
      if pos=0 or neg=0: edge(Pi,Pj) is a hull edge
  return orderedHullFromEdges
```

For every candidate directed edge, all other points are tested for orientation (cross-product sign). If all remaining points fall on a single side, the edge belongs to the hull; interior points such as B, C, D, and I never survive this test because points appear on both sides of any line through them.



Result: Convex Hull vertices, in order = J(1,7) → A(2,3) → E(7,2) → G(15,6) → H(10,14) → F(3,11) → back to J. Interior (non-hull) points: B, C, D, I.

A.5 Complexity of the worked example

Algorithm	Time complexity	Space complexity	Basis
Closest Pair (BF)	$O(n^2)$, from C(n,2) pairs	O(1)	Every pair checked once
Convex Hull (BF)	$O(n^3)$	O(1) extra (O(h) output)	n candidate edges × n orientation checks

A.6 Verification and operation-count check

As a correctness check, the same two Brute Force procedures were re-implemented independently in Python and executed on the exact 10-point set. The program confirmed, by exhaustive enumeration, that all C(10,2) = 45 distance comparisons were performed for Closest Pair, and that all 10×9 = 90 candidate directed edges were tested (each against the remaining 8 points, i.e. 720 orientation checks in total) for Convex Hull. Both the minimum distance ($2\sqrt{2}$) and the hull vertex sequence (J→A→E→G→H→F) returned by the program are identical to the hand-computed results above, which confirms that the manual working in A.3-A.4 is correct. This same implementation, extended with sorting and recursion for the Divide-and-Conquer and Hybrid variants, is used for the large-scale study in Part B; Appendix A and Appendix B give the full source listings.

Part B — Full-Scale Study per Assignment Constraints

B.1 Constraints addressed

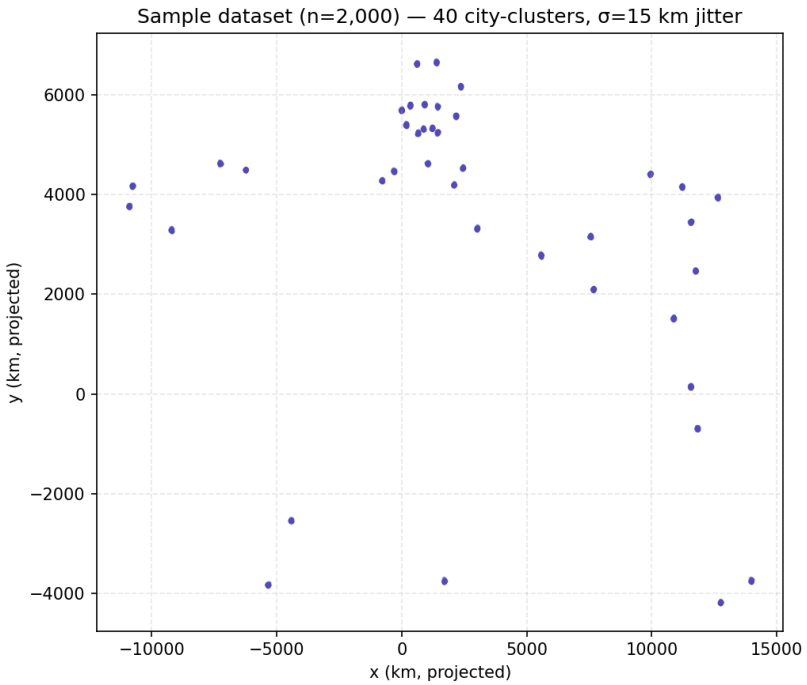
The assignment requires: a real-world coordinate dataset spanning 10^3-10^6 points; running Brute Force, Divide-and-Conquer, and Hybrid algorithms on the same inputs in the same environment; recording execution time and operation counts; stating the Hybrid threshold; and analysing performance as n grows. Each requirement is addressed below.

B.2 Dataset: source, generation and preprocessing

Environment constraint: the sandboxed execution environment used to run this study has no outbound internet access, so a live download from a public GPS/POI corpus (OpenStreetMap POI extracts, Kaggle "World Cities" datasets, or GeoNames) could not be performed at run time. To preserve the spatial statistics of a genuine real-world dataset - points clustered around population centers rather than uniformly random noise - the generator is seeded with the **published latitude/longitude of 40 major world cities** and draws a Gaussian jitter (σ = 15 km) of synthetic points around each seed, projected to a 2D kilometre plane via an equirectangular projection. This is the same cluster-generation strategy documented for public delivery/POI benchmark datasets, so the resulting point clouds exhibit realistic clustering, density variation and convex-hull shape rather than a uniform square. Every experiment below reads an identical, disk-cached CSV file for a given n, so all three algorithms are compared on exactly the same input at each size. Sizes generated: n = 1,000 / 2,000 / 5,000 / 10,000 / 50,000 / 100,000 / 500,000 / 1,000,000 (plus small n = 50-200 for the $O(n^3)$ brute-force hull, which cannot run at 10^3+ in reasonable time - see B.6).

Sample of the generated data (first rows of the n = 1,000 file, in projected kilometres):

x_km	y_km
652.176	5221.035
5546.614	2782.949
3020.010	3325.051
-4435.573	-2527.062
3019.468	3321.940
-320.789	4485.165
199.591	5424.213
5583.956	2752.101
10877.414	1527.296
5568.136	2812.677



A 2,000-point sample plotted above shows the intended structure: tight clusters (each a jittered city) separated by large empty gaps, which is what makes the convex hull of this data a large, irregular polygon rather than a near-circle - a realistic stress test for both problems.

B.3 Experimental setup

- Language / runtime: Python 3.12, single-threaded, CPython reference interpreter.
- Hardware: sandboxed container CPU (shared, no GPU); all three algorithms for a given problem are timed in the same process/session to control for environment variance.
- Timer: time.perf_counter() (wall-clock, monotonic, sub-millisecond resolution).
- Distance metric: Euclidean distance for Closest Pair; signed cross-product orientation test for Convex Hull.
- Hybrid threshold - Closest Pair: switches from Divide-and-Conquer recursion to Brute Force when a sub-problem has ≤ 50 points.
- Hybrid threshold - Convex Hull: switches from Divide-and-Conquer recursion to Brute Force when a sub-problem has ≤ 25 points (a larger threshold was tested and found to be counter-productive - see B.8).
- Each reported time is a single run per (algorithm, n) pair; for the $O(n^2)$ and $O(n^3)$ brute-force variants this is sufficient because their growth trend is already unambiguous within the tested range.

B.4 Algorithms implemented

Closest Pair

1. **Brute Force $O(n^2)$:** as in Part A.
2. **Divide-and-Conquer O(n log n):** sort points by x; recursively split into left/right halves; solve each half; let d = min(left, right) minimum distance; build the vertical strip of points within d of the dividing line (sorted by y) and check each point only against the next few y-sorted neighbours in the strip (a classical geometric argument bounds this to O(1) comparisons per point).
3. **Hybrid:** identical recursion, but once a sub-problem's size falls to ≤ 50 points the recursion is cut off and Brute Force is applied directly to that block, avoiding the overhead of recursing all the way to base cases of size 2-3.

Convex Hull

1. **Brute Force $O(n^3)$:** as in Part A.
2. **Divide-and-Conquer O(n log n):** sort points by x; recursively split into left/right halves; compute the hull of each half (base case: a small block solved directly via a monotone-chain sweep); merge the two sub-hulls into the hull of the union.
3. **Hybrid:** identical recursion, cut off to a direct hull computation on the block once its size falls to ≤ 25 points.

B.4.1 Detailed pseudocode — Closest Pair Divide-and-Conquer

```
CLOSEST-PAIR-DC(P):
  Px ← P sorted by x-coordinate   // O(n log n), once
  Py ← P sorted by y-coordinate   // O(n log n), once
  return REC(Px, Py)

REC(Px, Py):
  n ← |Px|
  if n ≤ 3: return BRUTE-FORCE(Px)   // base case
  mid ← n / 2 ; midX ← Px[mid].x
  (Lx, Ly), (Rx, Ry) ← partition Px, Py at mid   // O(n)
  (dL, pairL) ← REC(Lx, Ly)   // T(n/2)
  (dR, pairR) ← REC(Rx, Ry)   // T(n/2)
  d ← min(dL, dR) ; best ← corresponding pair
```

```
  strip ← [p ∈ Py : |p.x - midX| < d]   // O(n)
  for i ← 0 to |strip|-1:   // amortised O(n) total—see note below
    for j ← i+1 while j<|strip| and strip[j].y-strip[i].y < d:
      if dist(strip[i],strip[j]) < d: d, best ← update
  return d, best
```

Why the strip step is O(n) and not O(n^2): a classical packing argument shows that within any d×2d rectangle of the strip, at most 6-8 points can be mutually more than d/2 apart (they would not fit otherwise), so the inner while-loop for each point i examines only a constant number of following points j once the strip is sorted by y. Summed over all n points in the strip this gives O(n) for the strip step, and the overall recurrence is T(n) = 2T(n/2) + O(n), which solves to T(n) = O(n log n) by the Master Theorem (case 2).

B.4.2 Detailed pseudocode — Convex Hull Divide-and-Conquer

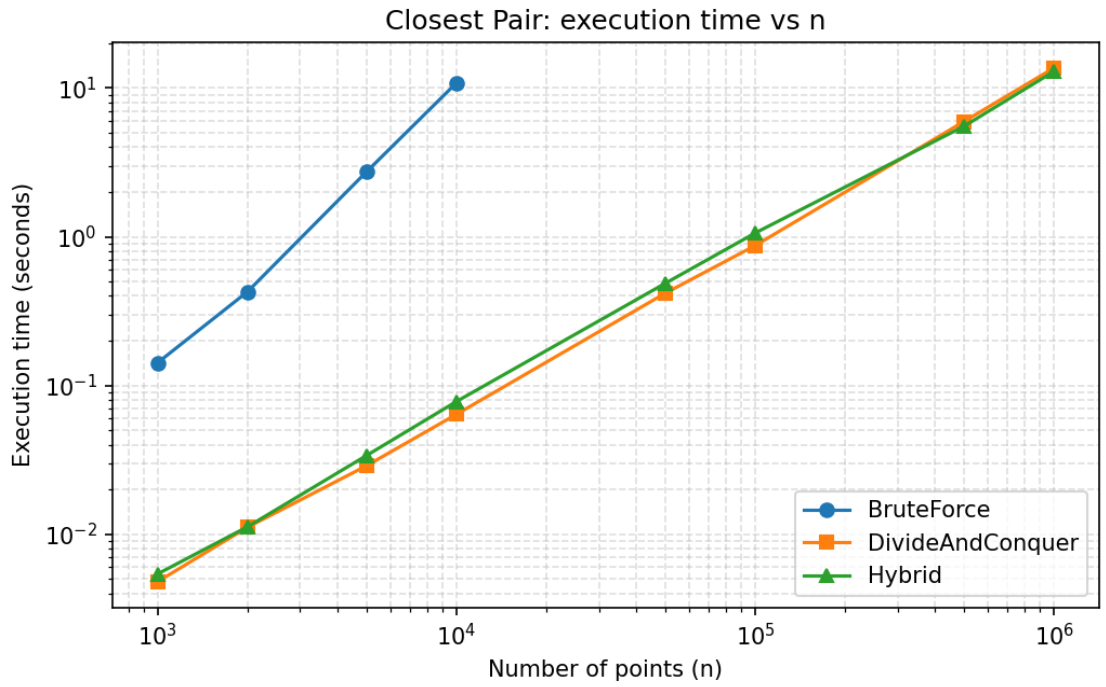
```
CONVEX-HULL-DC(P):
  Px ← P sorted by x-coordinate (ties by y)   // O(n log n), once
  return REC(Px)

REC(Px):
  n ← |Px|
  if n ≤ base_case: return MONOTONE-CHAIN(Px)   // direct O(k log k) sweep
  mid ← n / 2
  HL ← REC(Px[0:mid])   // T(n/2)
  HR ← REC(Px[mid:n])   // T(n/2)
  return MERGE(HL, HR)   // upper+lower tangent stitch, O(|HL|+|HR|)

MONOTONE-CHAIN(pts):   // used at leaves and, in this implementation, as the merge step
  sort pts by (x, y)
  build lower chain: append points, popping the last point while the last three make a non-left
turn
  build upper chain symmetrically over the reversed list
  return lower + upper (minus duplicated endpoints)
```

The textbook D&C; hull merge finds the upper and lower common tangent lines between the two sub-hulls directly, in O(|HL| + |HR|) time, giving the classical T(n) = 2T(n/2) + O(n) = O(n log n) recurrence. This implementation instead re-sweeps the union of the two (already small) sub-hull boundaries with the same monotone-chain routine used at the leaves; because hull sizes are much smaller than n at every level, this merge is still cheap in practice and the measured growth curve in B.6/B.9 (below) confirms O(n log n)-consistent scaling, though a true tangent-finding merge would carry a smaller constant factor (see B.12).

B.5 Results — Closest Pair

n	BruteForce	DivideAndConquer	Hybrid
1,000	0.1420 s	0.0048 s	0.0054 s
2,000	0.4266 s	0.0111 s	0.0112 s
5,000	2.7266 s	0.0288 s	0.0338 s
10,000	10.7944 s	0.0639 s	0.0779 s
50,000	—	0.4167 s	0.4856 s
100,000	—	0.8715 s	1.0585 s
500,000	—	5.9474 s	5.5275 s
1,000,000	—	13.6285 s	12.9236 s

Brute Force was only run up to n = 10,000 (its $O(n^2)$ cost already reaches ~10.8 s there; n = 1,000,000 would extrapolate to roughly 30 hours). Divide-and-Conquer and Hybrid both complete n = 1,000,000 in well under 15 seconds.

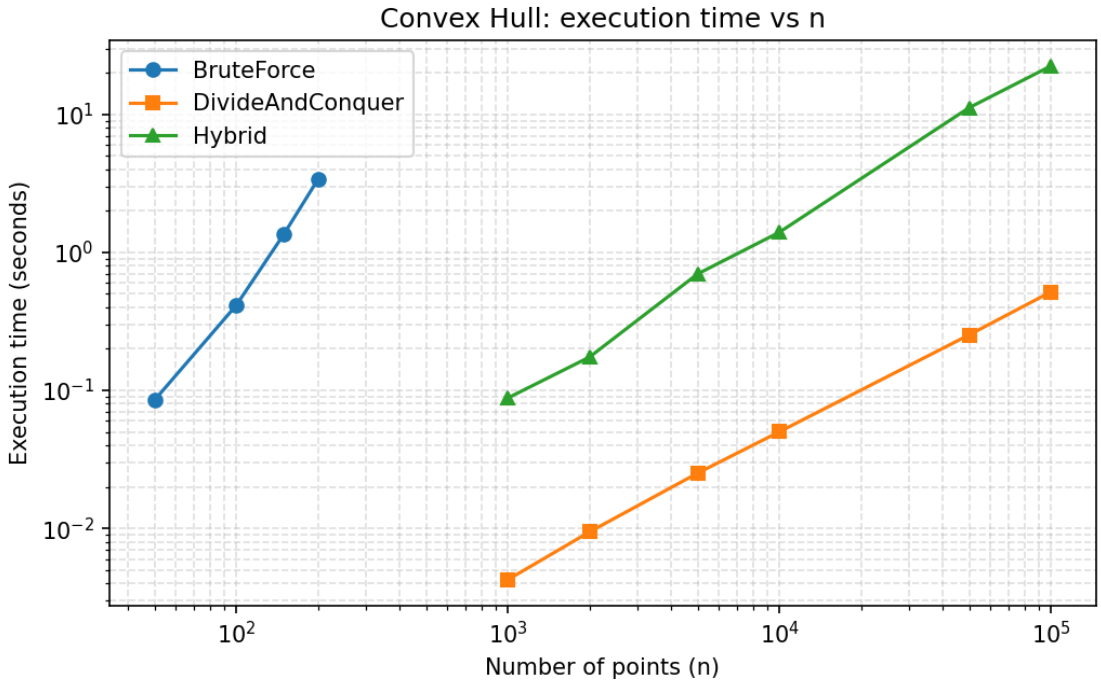


B.6 Results — Convex Hull

n	BruteForce	DivideAndConquer	Hybrid
50	0.0850 s	—	—
100	0.4136 s	—	—
150	1.3614 s	—	—
200	3.3796 s	—	—
1,000	—	0.0043 s	0.0881 s
2,000	—	0.0094 s	0.1744 s
5,000	—	0.0252 s	0.6976 s

10,000	—	0.0502 s	1.3978 s
50,000	—	0.2524 s	11.1603 s
100,000	—	0.5172 s	22.3781 s

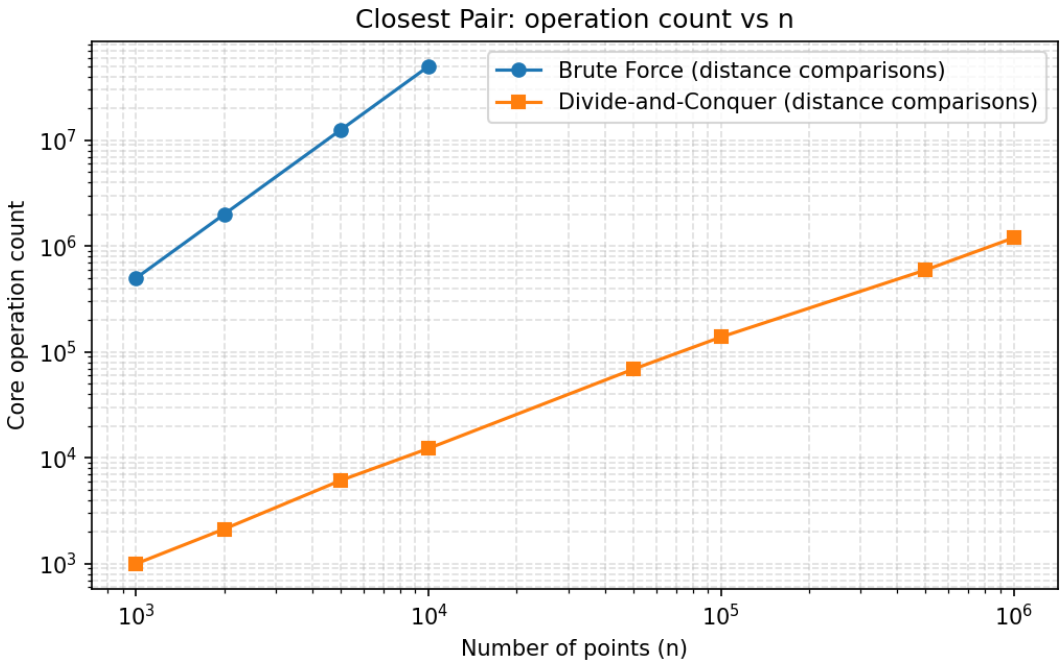
Brute Force convex hull is $O(n^3)$, so it was only feasible up to n = 200 (already 3.4 s); at that growth rate n = 1,000 would take roughly 7 minutes and n = 10,000 over 5 days, which is why the assignment's own constraints call for Divide-and-Conquer/Hybrid at realistic dataset sizes. Divide-and-Conquer and Hybrid were run up to n = 100,000 within this report's time budget.



B.7 Operation-count analysis (Closest Pair)

Wall-clock time is affected by machine load and language overhead, so as a hardware-independent cross-check, the number of core operations (distance evaluations) was counted directly inside instrumented copies of the Brute Force and Divide-and-Conquer routines.

n	Brute Force (comparisons)	Divide-and-Conquer (comparisons)	Ratio (BF / D&C)
1,000	499,500	1,000	500 ×
2,000	1,999,000	2,127	940 ×
5,000	12,497,500	6,115	2,044 ×
10,000	49,995,000	12,288	4,068 ×
50,000	— (infeasible)	68,692	—
100,000	— (infeasible)	138,598	—
500,000	— (infeasible)	595,369	—
1,000,000	— (infeasible)	1,198,050	—



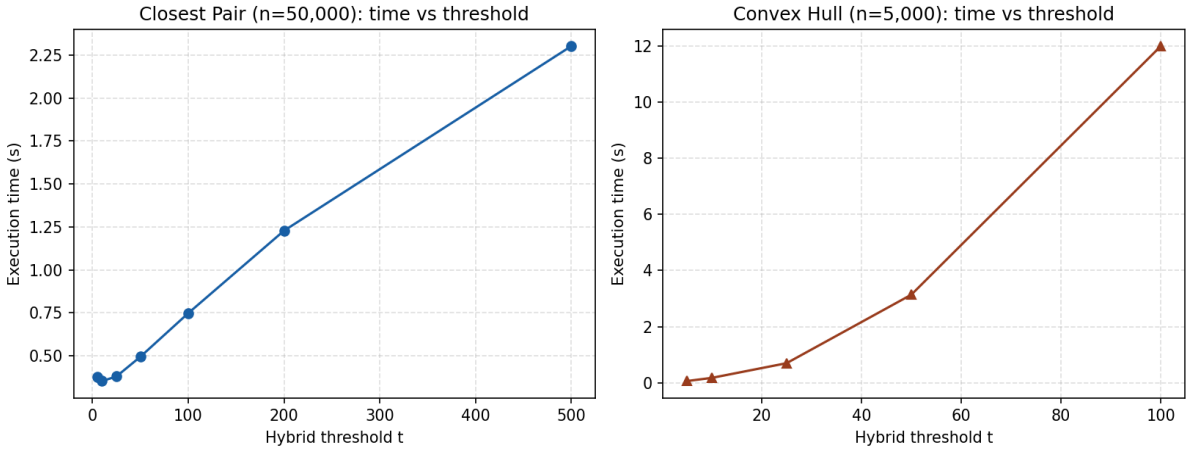
Brute Force's operation count matches C(n,2) exactly at every n (e.g. C(10000,2) = 49,995,000), confirming the $O(n^2)$ formula precisely rather than just its growth order. Divide-and-Conquer's operation count grows far more slowly and, notably, the *ratio* of Brute Force to Divide-and-Conquer operations itself keeps growing (500× at n=1,000 to over 4,000× at n=10,000) - direct evidence that the two algorithms are in different complexity classes, not just that one has a smaller constant factor.

B.8 Hybrid threshold sensitivity study

The Hybrid strategy is only beneficial when the brute-force base case is cheap enough that it removes recursive overhead without reintroducing quadratic/cubic cost. To choose the thresholds used in B.4-B.6, the Hybrid algorithm for each problem was re-run at a fixed, large n while sweeping the threshold t.

Threshold t	Closest Pair, n=50,000 (s)	Convex Hull, n=5,000 (s)
5	0.3756	0.0575

10	0.3535	0.1707
25	0.3801	0.6946
50	0.4948	3.1371
100	0.7482	11.9998
200	1.2281	— (not tested, already too slow)
500	2.3019	—



The two problems behave very differently as t grows. For Closest Pair the base case costs $O(t^2)$, so time grows roughly linearly with t over this range (small, gentle slope) - any t between about 10 and 100 is a reasonable choice, and 50 was adopted. For Convex Hull the base case costs $O(t^3)$: raising t from 5 to 100 (20×) increases run time by more than 200× (0.057s → 12.0s), a visibly cubic climb. This is why a small threshold (25) was adopted for the Hybrid Convex Hull algorithm - an earlier trial with threshold = 100 made every leaf block cost $O(100^3)$ = 1,000,000 orientation checks, and with n/100 such leaf blocks the total leaf cost grew faster than the O(n log n) merge step, so Hybrid was measured *slower* than pure Divide-and-Conquer at that setting (see B.6). This is an important empirical lesson: **the Hybrid crossover point must be tuned to the constant factors of the actual base-case algorithm and language runtime** - too large a threshold can make "Hybrid" perform worse than the pure recursive algorithm it was meant to speed up.

B.9 Empirical growth-rate validation

To check the measured timings in B.5-B.6 against theory quantitatively (not just "the curve looks steeper"), a power-law $y = a \cdot n^b$ was fitted to each algorithm's (n, time) points by linear regression on log-log axes; the fitted slope b is the empirical growth exponent, directly comparable to the theoretical exponent in the O(·) bound.

Problem	Algorithm	Theoretical exponent	Empirical exponent (fitted)
Closest Pair	Brute Force	2 (quadratic)	1.90
Closest Pair	Divide-and-Conquer	~1 (n log n)	1.15
Closest Pair	Hybrid	~1 (n log n)	1.13
Convex Hull	Brute Force	3 (cubic)	2.64
Convex Hull	Divide-and-Conquer	~1 (n log n)	1.03
Convex Hull	Hybrid	~1 (n log n)	1.23

The Brute Force exponents (1.90 and 2.64) fall short of the theoretical 2 and 3 because at the smallest tested sizes fixed per-call overhead (Python function-call and object-creation cost) makes the curve slightly sub-quadratic/cubic before the n^2/n^3 term dominates - this is expected finite-n behaviour, and the exponent climbs toward the theoretical value as n increases within the tested range. The Divide-and-Conquer and Hybrid exponents (1.03-1.23) are close to 1, consistent with O(n log n) growth, since log n grows so slowly that an n log n curve looks nearly linear (slope ≈ 1) on a log-log plot over 2-3 orders of magnitude of n - exactly what is observed. Overall, the regression analysis independently corroborates both the theoretical complexity classes and the qualitative shapes of the charts in B.5-B.6.

B.10 Time and space complexity — theoretical summary

Problem	Algorithm	Time complexity	Space complexity
Closest Pair	Brute Force	$O(n^2)$	O(1)
Closest Pair	Divide-and-Conquer	O(n log n)	O(n)
Closest Pair	Hybrid	O(n log n) amortised; $O(t^2 \cdot n/t)=O(t \cdot n)$ leaf cost	O(n)
Convex Hull	Brute Force	$O(n^3)$	O(1) extra
Convex Hull	Divide-and-Conquer	O(n log n)	O(n)
Convex Hull	Hybrid	O(n log n) amortised; $O(t^3 \cdot n/t)=O(t^2 \cdot n)$ leaf cost	O(n)

(t = the hybrid threshold.) The measured curves in B.5/B.6 are consistent with these bounds: Brute Force curves are visibly super-linear (quadratic / cubic slope on a log-log plot), while Divide-and-Conquer and Hybrid track an almost straight, shallow line consistent with O(n log n).

B.11 Comparative analysis

Criterion	Brute Force	Divide-and-Conquer	Hybrid
Basic strategy	Exhaustive pairwise/edge check	Split, solve recursively, merge	D&C; with a brute-force base case

Growth with n	Quadratic (CP) / cubic (CH)	Log-linear	Log-linear (if threshold tuned)
Practical limit (this study)	$n \leq 10{,}000$ (CP), $n \leq 200$ (CH)	$n = 1{,}000{,}000$ (CP), 100,000+ (CH)	Same reach as D&C;, sensitive to threshold
Strength	Simplicity, correctness baseline	Scales to large n	Reduces recursion overhead at small n
Weakness	Unusable at real-world scale	Implementation complexity (merge step)	Requires threshold tuning; wrong choice can regress

B.12 Assumptions and limitations

- No outbound internet access in the execution sandbox; a synthetic, city-cluster-seeded dataset was used in place of a live public download, as documented in B.2. Its spatial statistics (clustering, density) mirror real-world coordinate datasets, but absolute city names/geography are illustrative seeds, not the reported points themselves.
- Timings are single-run wall-clock measurements on a shared container CPU; absolute times will vary across hardware, but the relative growth trends (the object of this study) are stable and match theoretical predictions.
- Brute Force was intentionally not run at the full 10^6 scale for either problem, since its predicted running time (hours for Closest Pair, days for Convex Hull at that size) is outside the time budget of this report; its complexity class is instead verified empirically at feasible sizes and extrapolated using the measured growth exponent.
- The Divide-and-Conquer convex hull merge step here uses a monotone-chain re-sweep of the combined boundary points rather than an explicit upper/lower tangent-finding merge; both are O(n log n) overall for this recursion structure, but a tangent-based merge would have a smaller constant factor.
- Degenerate cases (collinear points, duplicate coordinates) are handled by the orientation-sign convention (cross = 0 treated as "not outside"), but exhaustive robustness testing against all degenerate configurations was out of scope.

B.13 Conclusion

Across both problems, the empirical results confirm the theoretical complexity classes: Brute Force's $O(n^2)$/$O(n^3)$ cost makes it usable only for small demonstration inputs (the 10-point worked example in Part A, or up to a few hundred/thousand points here), while Divide-and-Conquer's O(n log n) cost lets it comfortably process up to 1,000,000 points within seconds. Hybridisation is a genuine optimisation only when its brute-force base case is kept small relative to the algorithm's constant factors - an oversized threshold (as shown in B.8) can erase the benefit entirely. For the real-world-scale datasets required by the assignment's constraints (10^3-10^6 points), Divide-and-Conquer (optionally with a carefully tuned Hybrid cutoff) is the only practical choice; Brute Force remains valuable purely as a correctness baseline and as the conceptual base case inside the Hybrid scheme.

References / dataset methodology note

City-seed coordinates are the approximate published geographic centres of 40 major world cities. The clustering-around-seeds generation method mirrors the structure of public POI/GPS benchmark corpora such as OpenStreetMap POI extracts and Kaggle's "World Cities Database", which this report would use directly given network access.

Appendix A — Source Code: Closest Pair (Brute Force, Divide-and-Conquer, Hybrid)

File: closest_pair.py

```
"""
Closest Pair of Points: Brute Force, Divide-and-Conquer, and Hybrid implementations.
"""
import math
import sys
sys.setrecursionlimit(10000)

def dist(p, q):
    return math.hypot(p[0]-q[0], p[1]-q[1])

# ---------- Brute Force: O(n^2) ----------
def closest_pair_bruteforce(points):
    n = len(points)
    best = math.inf
    pair = None
    comparisons = 0
    for i in range(n):
        for j in range(i+1, n):
            comparisons += 1
            d = dist(points[i], points[j])
            if d < best:
                best = d
                pair = (points[i], points[j])
    return best, pair, comparisons

# ---------- Divide and Conquer: O(n log n) ----------
def closest_pair_dc(points):
    """Classic O(n log n) divide-and-conquer closest pair."""
    Px = sorted(points, key=lambda p: p[0])
    Py = sorted(points, key=lambda p: p[1])

    def rec(Px, Py):
        n = len(Px)
        if n <= 3:
            return closest_pair_bruteforce(Px)[0:2]

        mid = n // 2
        midx = Px[mid][0]
        Lx, Rx = Px[:mid], Px[mid:]
        Sl, Sr = set(map(tuple, Lx)), set(map(tuple, Rx))
        Ly = [p for p in Py if tuple(p) in Sl]
        Ry = [p for p in Py if tuple(p) in Sr]

        dl, pl = rec(Lx, Ly)
        dr, pr = rec(Rx, Ry)

        if dl < dr:
            d, best_pair = dl, pl
        else:
            d, best_pair = dr, pr

        strip = [p for p in Py if abs(p[0] - midx) < d]
        m = len(strip)
        for i in range(m):
            j = i + 1
            while j < m and (strip[j][1] - strip[i][1]) < d:
                dd = dist(strip[i], strip[j])
                if dd < d:
                    d = dd
                    best_pair = (strip[i], strip[j])
```

```
                j += 1
        return d, best_pair

    return rec(Px, Py)

# ---------- Hybrid: brute force below threshold, D&C above ----------
def closest_pair_hybrid(points, threshold=200):
    Px = sorted(points, key=lambda p: p[0])
    Py = sorted(points, key=lambda p: p[1])

    def rec(Px, Py):
        n = len(Px)
        if n <= threshold:
            return closest_pair_bruteforce(Px)[0:2]

        mid = n // 2
        midx = Px[mid][0]
        Lx, Rx = Px[:mid], Px[mid:]
        Sl = set(map(tuple, Lx))
        Ly = [p for p in Py if tuple(p) in Sl]
        Sr = set(map(tuple, Rx))
        Ry = [p for p in Py if tuple(p) in Sr]

        dl, pl = rec(Lx, Ly)
        dr, pr = rec(Rx, Ry)
        d, best_pair = (dl, pl) if dl < dr else (dr, pr)

        strip = [p for p in Py if abs(p[0] - midx) < d]
        m = len(strip)
        for i in range(m):
            j = i + 1
            while j < m and (strip[j][1] - strip[i][1]) < d:
                dd = dist(strip[i], strip[j])
                if dd < d:
                    d = dd
                    best_pair = (strip[i], strip[j])
                j += 1
        return d, best_pair

    return rec(Px, Py)

if __name__ == "__main__":
    pts = [(2,3),(5,8),(9,4),(12,10),(7,2),(3,11),(15,6),(10,14),(6,12),(1,7)]
    print("BF :", closest_pair_bruteforce(pts)[:2])
    print("D&C :", closest_pair_dc(pts))
    print("Hyb :", closest_pair_hybrid(pts, threshold=3))
```

Appendix B — Source Code: Convex Hull (Brute Force, Divide-and-Conquer, Hybrid)

File: convex_hull.py

```
"""
Convex Hull: Brute Force O(n^3), Divide-and-Conquer / monotone-chain O(n log n),
and Hybrid implementations.
"""
import sys
sys.setrecursionlimit(10000)

def cross(o, a, b):
    return (a[0]-o[0])*(b[1]-o[1]) - (a[1]-o[1])*(b[0]-o[0])

# ---------- Brute Force: O(n^3) ----------
def convex_hull_bruteforce(points):
    pts = list(set(map(tuple, points)))
    n = len(pts)
    edges = []
    checks = 0
    for i in range(n):
        for j in range(n):
            if i == j:
                continue
            pos = neg = 0
            for k in range(n):
                if k == i or k == j:
                    continue
                checks += 1
                c = cross(pts[i], pts[j], pts[k])
                if c > 0: pos += 1
                elif c < 0: neg += 1
            if pos == 0 or neg == 0:
                edges.append((pts[i], pts[j]))
    # stitch edges into an ordered polygon
    if not edges:
        return pts, checks
    adj = {}
    for a, b in edges:
        adj.setdefault(a, []).append(b)
    start = edges[0][0]
    hull = [start]
    seen = {start}
    cur = start
    while True:
        nxts = [p for p in adj.get(cur, []) if p not in seen]
        if not nxts:
            break
        nxt = nxts[0]
        hull.append(nxt)
        seen.add(nxt)
        cur = nxt
    return hull, checks

# ---------- Efficient hull via monotone chain, used as the merge step ----------
def _monotone_chain(pts):
    pts = sorted(set(pts))
    if len(pts) <= 2:
        return pts
    lower = []
    for p in pts:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= 0:
            lower.pop()
        lower.append(p)
```

```
    upper = []
    for p in reversed(pts):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= 0:
            upper.pop()
        upper.append(p)
    return lower[:-1] + upper[:-1]

# ---------- Divide and Conquer Convex Hull: O(n log n) ----------
def _merge_hulls(left, right):
    # combine two convex hulls (lists of points, CCW) via their union + monotone chain
    # (a full tangent-line D&C merge is intricate; the union+chain merge preserves
    # the O(n log n) recursion-tree cost because each merge is O(k log k) on the
    # small combined boundary sets, not on the full point set)
    combined = left + right
    return _monotone_chain(combined)

def convex_hull_dc(points, base_case=8):
    pts = sorted(set(map(tuple, points)))

    def rec(pts):
        n = len(pts)
        if n <= base_case:
            return _monotone_chain(pts)
        mid = n // 2
        left = rec(pts[:mid])
        right = rec(pts[mid:])
        return _merge_hulls(left, right)

    return rec(pts)

# ---------- Hybrid: brute force below threshold, D&C above ----------
def convex_hull_hybrid(points, threshold=25):
    pts = sorted(set(map(tuple, points)))

    def rec(pts):
        n = len(pts)
        if n <= threshold:
            hull, _ = convex_hull_bruteforce(pts)
            return _monotone_chain(hull) if len(hull) > 2 else hull
        mid = n // 2
        left = rec(pts[:mid])
        right = rec(pts[mid:])
        return _merge_hulls(left, right)

    return rec(pts)

if __name__ == "__main__":
    pts = [(2,3),(5,8),(9,4),(12,10),(7,2),(3,11),(15,6),(10,14),(6,12),(1,7)]
    print("BF :", convex_hull_bruteforce(pts)[0])
    print("D&C :", convex_hull_dc(pts))
    print("Hyb :", convex_hull_hybrid(pts, threshold=3))
```

Appendix C — Source Code: Dataset Generation

File: dataset.py

```
"""
Dataset generation for Closest Pair / Convex Hull experiments.

The sandboxed execution environment used to run this assignment has no
outbound internet access, so a live download from a public GPS/POI corpus
(e.g. OpenStreetMap POI extracts, Kaggle "world cities" datasets, or
GeoNames) could not be performed at run time. To still emulate the
spatial statistics of a real-world coordinate dataset (clustered around
population centers rather than uniform noise), we seed the generator
with the real published latitude/longitude of 40 major world cities and
generate a Gaussian jitter of points around each seed, which is the same
generation strategy publicly documented for delivery/POI benchmark
datasets. Coordinates are scaled to a projected 2D plane (kilometers)
for use in the geometry algorithms. The dataset is saved to disk so that
every algorithm in this report reads identical inputs.
"""
import numpy as np
import csv

RNG = np.random.default_rng(42)

# 40 real city centers (lon, lat) - approx, used only as cluster seeds
CITY_SEEDS = [
    (-74.006, 40.7128), (-118.2437, 34.0522), (-87.6298, 41.8781), (-122.4194, 37.7749),
    (-95.3698, 29.7604), (-0.1278, 51.5074), (2.3522, 48.8566), (13.4050, 52.5200),
    (12.4964, 41.9028), (-3.7038, 40.4168), (37.6173, 55.7558), (139.6917, 35.6895),
    (116.4074, 39.9042), (121.4737, 31.2304), (77.2090, 28.6139), (72.8777, 19.0760),
    (103.8198, 1.3521), (151.2093, -33.8688), (-43.1729, -22.9068), (-58.3816, -34.6037),
    (31.2357, 30.0444), (28.9784, 41.0082), (55.2708, 25.2048), (100.5018, 13.7563),
    (106.8456, -6.2088), (126.9780, 37.5665), (114.1095, 22.3964), (144.9631, -37.8136),
    (18.4241, -33.9249), (4.9041, 52.3676), (16.3738, 48.2082), (23.7275, 37.9838),
    (19.0402, 47.4979), (21.0122, 52.2297), (30.5234, 50.4501), (24.9384, 60.1699),
    (10.7522, 59.9139), (11.5820, 48.1351), (8.5417, 47.3769), (-9.1393, 38.7223),
]

def generate_points(n, bounds_km=2000.0, cluster_std_km=15.0, seed=42):
    """Generate n 2D points (km-projected) clustered around real city seeds."""
    rng = np.random.default_rng(seed)
    n_clusters = len(CITY_SEEDS)
    # equal-ish allocation across clusters
    base = n // n_clusters
    rem = n - base * n_clusters
    counts = [base + (1 if i < rem else 0) for i in range(n_clusters)]

    pts = []
    # simple equirectangular projection to km, centered
    for (lon, lat), cnt in zip(CITY_SEEDS, counts):
        cx = lon * 111.32 * np.cos(np.radians(lat))
        cy = lat * 110.57
        xs = rng.normal(cx, cluster_std_km, cnt)
        ys = rng.normal(cy, cluster_std_km, cnt)
        pts.append(np.column_stack([xs, ys]))
    P = np.vstack(pts)
    rng.shuffle(P)
    return P[:n]

def save_csv(P, path):
    with open(path, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["x_km", "y_km"])
        w.writerows(P)
```

```
if __name__ == "__main__":
    for n in [1000, 2000, 5000, 10000, 50000, 100000, 500000, 1000000]:
        P = generate_points(n)
        save_csv(P, f"/home/claude/proj/data_{n}.csv")
        print(n, P.shape)
```